BIỂU DIỄN ĐỒ THỊ

Danh sách cung:

```
#include <stdio.h>
#define MAX_N 100
#define MAX_M 500
```

1. Khai báo cấu trúc cạnh Edge

```
typedef struct{
	int u, v;
}Edge;
```

2. Khai báo cấu trúc Graph

```
typedef struct{
	Edge edges[MAX_M];
	int n, m;
}Graph;
```

3. Khởi tạo đồ thị

```
void init_graph(Graph *pG, int n){
	pG->n = n;
	pG->m = 0;
}
```

4. Thêm cung vào

Đồ thị có hướng

```
//Thêm thông thường
void add_edge(Graph *pG, int u, int v){
		pG->edges[pG->m].u = u;
		pG->edges[pG->m].v = v;
		pG->m++;
}
//Kiểm tra hợp lệ
void add_edge(Graph *pG, int u, int v){
	if(u >= 1 && v >= 1 && u <=
pG->n && v <= pG->n){
		pG->edges[pG->m].u = u;
		pG->edges[pG->m].v = v;
		pG->m++;
	}
}
```

Đồ thị vô hướng

```
//Thêm thông thường
void add_edge(Graph *pG, int u, int v){
		pG->edges[pG->m].u = u;
		pG->edges[pG->m].v = v;
		pG->m++;
}

//Kiểm tra hợp lệ
void add_edge(Graph *pG, int u, int v){
	if(u >= 1 && v >= 1 && u <=
pG->n && v <= pG->n){
		pG->edges[pG->m].u = u;
		pG->edges[pG->m].v = v;
		pG->m++;
	}
}
```

```
//Kiểm tra (u, v) đã tồn tại -> không thêm

void add_edge(Graph *pG, int u, int v){
	for(int e = 0; e < pG->m; e++)
		if((pG->edges[e].u == u
&& pG->edges[e].v == v))
				return;
	if (u >= 1 && v >= 1 && u <=
pG->n && v <= pG->n){
		pG->edges[pG->m].u = u;
		pG->edges[pG->m].v = v;
		pG->m++;
	}
}
```

```
//Kiểm tra (u, v) hoặc (v, u) đã tồn tại ->
không thêm
void add_edge(Graph *pG, int u, int v){
	for(int e = 0; e < pG->m; e++){
		if((pG->edges[e].u == u
&& pG->edges[e].v == v)
			|| (pG->edges[e].u
== v && pG->edges[e].v == u))
		return;
	}
	if(u >= 1 && v >= 1 && u <=
pG->n && v <= pG->n){
		pG->edges[pG->m].u = u;
		pG->edges[pG->m].v = v;
		pG->m++;
	}
}
```

5. Kiểm tra u và v kề nhau:

Đồ thị có hướng

```
int adjacent(Graph *pG, int u, int v){
	for(int e = 0; e < pG->m; e++){
		if(pG->edges[e].u == u
&& pG->edges[e].v == v)
			return 1;
	}
	return 0;
}
```

Đồ thị vô hướng

```
int adjacent(Graph *pG, int u, int v){
	for(int e = 0; e < pG->m; e++){
		if((pG->edges[e].u == u
&& pG->edges[e].v == v)
			|| (pG->edges[e].u
== v && pG->edges[e].v == u))
				return 1;
	}
	return 0;
}
```

6. Tính bậc của một đỉnh:

Đồ thị có hướng

```
int degree(Graph *pG, int u){
	int deg_u = 0;
	for(int e = 0; e <= pG->m; e++){
		if(pG->edges[e].u == u)
			deg_u++;
		if(pG->edges[e].v == u)
			deg_u++;
	}
	return deg_u;
}
```

Đồ thị vô hướng

```
int degree(Graph *pG, int u){
	int deg_u = 0;
	for(int e = 0; e <= pG->m; e++){
		if(pG->edges[e].u == u)
			deg_u++;
		if(pG->edges[e].v == u)
			deg_u++;
	}
	return deg_u;
}
```

7. In danh sách kề của một đỉnh:

```
void neighbours(Graph *pG, int u){
	printf("Neighbour[%d] = ", u);
	for(int v = 1; v <= pG->n; v++){
		if(adjacent(pG, u, v))
			printf("%d ", v);
	}
```

```
	printf("\n");
}
```

Ma trận kề (Ma trận đỉnh đỉnh)

```
#include <stdio.h>
#define MAX_N 100
```

1. Khai báo cấu trúc Graph

```
typedef struct{
	int A[MAX_N][MAX_N];
	int n, m;
}Graph;
```

2. Khởi tạo đồ thị

```
void init_graph(Graph *pG, int n){
	pG->n = n;
	pG->m = 0;
	for(int i = 1; i <= n; i++)
		for(int j = 1; j <= n; j++)
			pG->A[i][j] = 0;
}
```

3. Thêm cung vào

Đồ thị có hướng

```
void add_edge(Graph *pG, int u, int v){
	pG->A[u][v]++;
	pG->m++;
}
```

Đồ thị vô hướng

```
void add_edge(Graph *pG, int u, int v){
	if(u != v){
		pG->A[u][v]++;
		pG->A[v][u]++;
	}
	else pG->A[u][v]++;
	pG->m++;
}
```

4. Tính bậc của một đỉnh:

Đồ thị có hướng

```
int degree(Graph *pG, int u){
	int deg_u = 0;
	for(int v = 1; v <= pG->n; v++){
		if(u == v) deg_u += 2*
pG->A[u][v];
		else deg_u += pG-
>A[u][v];
	}
	return deg_u;
}
```

Đồ thị vô hướng

```
int degree(Graph *pG, int u){
	int deg_u = 0;
	for(int v = 1; v <= pG->n; v++){
		deg_u += (pG->A[u][v] +
pG->A[v][u]);
	}
	return deg_u;
}

int indegree(Graph *pG, int u){
	int indeg_u = 0;
	for(int v = 1; v <= pG->n; v++){
		indeg_u += pG->A[v][u];
	}
	return indeg_u;
```

```
}

int outdegree(Graph *pG, int u){
	int outdeg_u = 0;
	for(int v = 1; v <= pG->n; v++){
		outdeg_u += pG->A[u][v];
	}
	return outdeg_u;
}
```

5. In ra danh sách kề của một đỉnh:

Đồ thị có hướng

```
void neighbours(Graph *pG, int u){
	printf("neighbours(%d) = ", u);
	for(int v = 1; v <= pG->n; v++)
		if(pG->A[u][v])
			printf("%d ", v);
	printf("\n");
}
```

Đồ thị vô hướng

```
void neighbours(Graph *pG, int u){
	printf("neighbours(%d) = ", u);
	for(int v = 1; v <= pG->n; v++)
		if(pG->A[u][v] || pG->A[v][u])
			printf("%d ", v);
	printf("\n");
}
```

Danh sách kề

(cần khai báo thêm cấu trúc List)

```
#include <stdio.h>
#define MAX_N 100
```

1. Khai báo cấu trúc Graph

```
typedef struct {
	int n;
	List adj[MAX_N];
}Graph;
```

2. Khởi tạo đồ thị

```
void init_graph(Graph *pG, int n){
	pG->n = n;
	for(int i = 1; i <= n; i++){
		make_null_List(&pG->adj[i]);
	}
}
```

3. Thêm cung vào

Đồ thị có hướng

```
void add_edge(Graph *pG, int u, int v){
	push_back(&pG->adj[u], v);
}
```

Đồ thị vô hướng

```
void add_edge(Graph *pG, int u, int v){
	if (u!=v){
		push_back(&pG->adj[u], v);
		push_back(&pG->adj[v], u);
	}
```

```
        else push_back(&pG->adj[u], v);
}
```

5. Kiểm tra u và v kề nhau:

```
int adjacent(Graph *pG, int u, int v){
        for(int i = 1; i <= pG->adj[u].size; i++){
                if(v == element_at(&pG->adj[u], i))
                        return 1;
        }
        return 0;
}
```

6. In danh sách kề của một đỉnh:

```
void neighbours(Graph *pG, int u){
        printf("neighbour[%d] = ", u);
        for(int i = 1; i <= pG->adj[u].size; i++){
                printf("%d ", element_at(&pG->adj[u], i));
        }
        printf("\n");
}
```

7. Tính bậc của một đỉnh

Đồ thị có hướng

```
int indegree(Graph *pG, int u){
        int indeg_u = 0;
        for(int i = 1; i <= pG->n; i++){
                List L = pG->adj[i];
                for(int j = 1; j <= L.size;
j++)
                        if(element_at(&L,
j) == u)
                                indeg_u++;
        }
        return indeg_u;
}

int outdegree(Graph *pG, int u){
        return pG->adj[u].size;
}

int degree(Graph *pG, int u){
        return indegree(pG, u) +
outdegree(pG, u);
}
```

Đồ thị vô hướng

```
int degree(Graph *pG, int u){
        return pG->adj[u].size;
}
```

Chuyển Ma Trận Kề Thành Ds Cung:

```
#include <stdio.h>
#define MAX_N 100
```

```
int main(){
	int A[MAX_N][MAX_N], n;
	scanf("%d", &n);
	for(int i = 1; i <= n; i++)
		for(int j = 1; j <= n; j++)
			scanf("%d",&A[i][j]);
	for(int i = 1; i <= n; i++)
		for(int j = i; j <= n; j++)
			if(A[i][j] != 0){
				int t = 0;
				while (t < A[i][j]){
					printf("%d %d\n", i, j);
					t++;
				}
			}
	return 0;
}
```

Chuyển Ma Trận Kề Thành Ds Kề:

```
#include <stdio.h>
#define MAX_N 100

int main(){
	int A[MAX_N][MAX_N];
	int n;
	scanf("%d", &n);
	for(int i = 1; i <= n; i++)
		for(int j = 1; j <= n; j++)
			scanf("%d", &A[i][j]);
	for(int i = 1; i <= n; i++){
		for(int j = 1; j <= n; j++){
			if(A[i][j]!=0){
				int t = 0;
				while (t < A[i][j]){
					printf("%d ", j);
					t++;
				}
			}
		}
		printf("0\n");
	}
	return 0;
}
```

Chuyển Ds Kề Thành Ma Trận Kề:

```
#include <stdio.h>
#define MAX_N 100

int main(){
```

```
	int n, A[MAX_N][MAX_N];
	scanf("%d", &n);
	for(int i = 1; i <= n; i++)
		for(int j = 1; j <= n; j++)
			A[i][j] = 0;
	for(int u = 1; u <= n; u++){
		int v;
		do{
			scanf("%d", &v);
			if(v != 0) A[u][v]++;
		}while(v != 0);
	}
	for(int i = 1; i <= n; i++){
		for(int j = 1; j <= n; j++)
			printf("%d ", A[i][j]);
		printf("\n");
	}
	return 0;
}
```

DUYỆT ĐỒ THỊ & ỨNG DỤNG

BFS:

```
int mark[MAX_N];
void BFS(Graph *pG, int s){
	Queue Q; make_null_Queue(&Q);
	enqueue(&Q, s);
	while(!empty_Queue(&Q)){
		int u = top(&Q); pop(&Q);
		if(mark[u]) continue;
		printf("%d\n", u);
		mark[u] = 1;
		for(int v = 1; v <= pG->n; v++){
			if(pG->A[u][v])
				enqueue(&Q, v);
		}
	}
}
```

Dựng cây duyệt đồ thị BFS:

```
int mark[MAX_N];
int parent[MAX_N];
void BFS(Graph *pG, int s, int p){
	Queue Q; make_null_queue(&Q);
	enqueue(&Q, s);
	while(!empty_Queue(&Q)){
		int u = top(&Q); pop(&Q);
		if(mark[u]) continue;
		mark[u] = 1;
		for(int v = 1; v <= pG->n; v++){
```

```
                        if(pG->A[u][v] && !mark[v]){
                                enqueue(&Q, v);
                                if(parent[v] == -1)
                                        parent[v] = u;
                        }
                }
        }
}

int main(){
        int n, m;
        scanf("%d%d", &n, &m);
        Graph G;
        init_graph(&G, n);
        for(int i = 0; i < m; i++){
                int u, v;
                scanf("%d%d", &u, &v);
                add_edge(&G, u, v);
        }
        for(int i = 1; i <= n; i++){
                parent[i] = -1;
        }
        for(int i = 1; i <= n; i++){
                if(!mark[i])
                        BFS(&G, i, -1);
        }
        for(int i = 1; i <= n; i++){
                printf("%d %d\n", i, parent[i]);
        }
        return 0;
}
```

DFS:

Sử dụng Stack

```
int mark[MAX_N];
void DFS(Graph *pG, int s){
        Stack S; make_null_Stack(&S);
        push(&S, s);
        while(!empty(&S)){
            int u = top(&S); pop(&S);
            if(mark[u]) continue;
             mark[u] = 1;
             printf("%d\n", u);
             for(int v = 1; v <= pG->n; v++){
                  if(pG->A[u][v])
                          push(&S, v);
                  }
        }
}
```

Đệ quy

```
int mark[MAX_N];
void DFS(Graph *pG, int u){
        if(mark[u])
                return;
        printf("%d\n", u);
        mark[u] = 1;
        for(int v = 1; v <= pG->n; v++)
                if(pG->A[u][v])
                        DFS(pG, v);
}
```

Note: Muốn thứ tự duyệt của DFS đệ quy giống Stack -> Quét v từ n -> 1 for(int v = pG->n; v >= 1; v--)

Dựng cây duyệt đồ thị DFS đệ quy:

```
int mark[MAX_N];
int parent[MAX_N];
void BFS(Graph *pG, int s, int p){
	Queue Q; make_null_queue(&Q);
	enqueue(&Q, s);
	while(!empty_Queue(&Q)){
		int u = top(&Q); pop(&Q);
		if(mark[u]) continue;
		mark[u] = 1;
		for(int v = 1; v <= pG->n; v++){
			if(pG->A[u][v] && !mark[v]){
				enqueue(&Q, v);
				if(parent[v] == -1)
					parent[v] = u;
			}
		}
	}
}

int main(){
	int n, m;
	scanf("%d%d", &n, &m);
	Graph G;
	init_graph(&G, n);
	for(int i = 0; i < m; i++){
		int u, v;
		scanf("%d%d", &u, &v);
		add_edge(&G, u, v);
	}
	for(int i = 1; i <= n; i++){
		parent[i] = -1;
	}
	for(int i = 1; i <= n; i++){
		if(!mark[i])
			BFS(&G, i, -1);
	}
	for(int i = 1; i <= n; i++){
		printf("%d %d\n", i, parent[i]);
	}
	return 0;
}
```

Liên thông

Kiểm tra đồ thị liên thông

//Hàm DFS đệ quy đã khai báo

```
int connected(Graph *pG){
	DFS(pG, 1);
	for(int i = 1; i <= pG->n; i++){
		if (!mark[i]) return 0;
	}
	return 1;
}

int main(){
	int n,m;
	scanf("%d%d", &n, &m);
	Graph G;
	init_graph(&G, n);
	for(int i = 0; i < m; i++){
		int u, v;
		scanf("%d%d", &u, &v);
		add_edge(&G, u, v);
	}
	if(connected(&G))
		printf("CONNECTED\n");
	else printf("DISCONNECTED\n");
	return 0;
}
```

Đếm số bộ phận liên thông

```
//Hàm DFS đệ quy

int main(){
	int n, m;
	scanf("%d%d", &n, &m);
	Graph G;
	init_graph(&G, n);
	for(int i = 0; i < m; i++){
		int u, v;
		scanf("%d%d", &u, &v);
		add_edge(&G, u, v);
	}
	int res = 0;
	for(int i = 1; i <= n; i++){
		if(!mark[i]){
			DFS(&G, i);
			res++;
		}
	}
	printf("%d", res);
	return 0;
}
```

Đếm số đỉnh bộ phận liên thông:

```
//Hàm DFS đệ quy
```

```
int res;

int main(){
	int n, m;
	scanf("%d%d", &n, &m);
	Graph G;
	init_graph(&G, n);
	for(int i = 0; i < m; i++){
		int u, v;
		scanf("%d%d", &u, &v);
		add_edge(&G, u, v);
	}
	int s; scanf("%d", &s);
	DFS(&G, s);
	printf("%d", res);
	return 0;
}
```

Bộ phận liên thông có số đỉnh lớn nhất

```
//Hàm DFS đệ quy

int cnt_v;

int main(){
	int n, m;
	scanf("%d%d", &n, &m);
	Graph G;
	init_graph(&G, n);
	for(int i = 0; i < m; i++){
		int u, v;
		scanf("%d%d", &u, &v);
		add_edge(&G, u, v);
	}
	int res = 0;
	for(int i = 1; i <= n; i++){
		if(!mark[i]){
			cnt_v = 0;
			DFS(&G, i);
			res = max(res, cnt_v);
		}
	}
	printf("%d", res);
	return 0;
}
```

Chu trình

Kiểm tra đồ thị có chu trình

* Đồ thị có hướng:

```
int mark[MAX_N];
```

```
int has_circle;
void DFS(Graph *pG, int u, int p){
	mark[u] = 1;
	for(int v = 1; v <= pG->n; v++) {
		if(pG->A[u][v]){
			if (!mark[v])
				DFS(pG, v, u);
			else if(mark[v] == 1)
				has_circle = 1;
		}
	}
	mark[u] = 2;
}
```

* Đồ thị vô hướng:

```
int mark[MAX_N], parent[MAX_N];
int has_circle, start, end;
void DFS(Graph *pG, int u, int p){
	mark[u] = 1;
   parent[u] = p;
	for(int v = 1; v <= pG->n; v++)
		if(pG->A[u][v]){
			if (v == p) continue;
			if (!mark[v])
				DFS(pG, v, u);
			else if(mark[v] == 1){
				has_circle = 1;
	   start = v;
	   end = u;
	}

		}
	mark[u] = 2;
}
```

* Hàm main

```
int main(){
	int n,m;
	scanf("%d%d", &n, &m);
	Graph G;
	init_graph(&G, n);
	for(int i = 0; i < m; i++){
		int u, v;
		scanf("%d%d", &u, &v);
		add_edge(&G, u, v);
	}
	for(int i = 1; i <= n; i++)
		if(!mark[i])
			DFS(&G, i,-1);
```

```
        if(has_circle) printf("CIRCLED");
        else printf("NO CIRCLE");
        return 0;
}
```

In ra chu trình hợp lệ

* Đồ thị có hướng

```
int mark[MAX_N];
int parent[MAX_N];
int has_circle, start, end;
void DFS(Graph *pG, int u, int p){
   parent[u] = p;
        mark[u] = 1;
        for(int v = 1; v <= pG->n; v++)
                if(pG->A[u][v]){
                        if (!mark[v])
                                DFS(pG, v, u);
                        else if(mark[v] == 1){
           start = v;
           end = u;
                                has_circle = 1;
        }
                }
        mark[u] = 2;
}

int main(){
        //freopen("dt.txt", "r", stdin);
        int n, m, u, v;
        scanf("%d%d", &n, &m);
        Graph G;
        init_graph(&G, n);
        for (int i = 0; i < m; i++){
                scanf("%d%d", &u, &v);
                add_edge(&G, u, v);
        }
        has_circle = 0;
        for(u = 1; u <= G.n; u++)
                if(!mark[u])
                        DFS(&G, u, -1);
        if(has_circle) {
                List L; make_null(&L);
      push_back(&L, start);
      while (end != start){
         push_back(&L, end);
         end = parent[end];
      }
      push_back(&L, start);
      for(int i = L.size; i >= 1; i--){
         printf("%d ", element_at(&L, i));
```

```
        }

        }
        else printf("-1");
        return 0;
}
```

* Đồ thị vô hướng

```
int mark[MAX_N], parent[MAX_N];
int has_circle, start, end;
void DFS(Graph *pG, int u, int p){
        mark[u] = 1;
    parent[u] = p;
        for(int v = 1; v <= pG->n; v++)
                if(pG->A[u][v]){
                        if (v == p) continue;
                        if (!mark[v])
                                DFS(pG, v, u);
                        else if(mark[v] == 1){
                                has_circle = 1;
            start = v;
            end = u;
        }

                }
        mark[u] = 2;
}

int main(){
        //freopen("dt.txt", "r", stdin);
        int n, m, u, v;
        scanf("%d%d", &n, &m);
        Graph G;
        init_graph(&G, n);
        for (int i = 0; i < m; i++){
                scanf("%d%d", &u, &v);
                add_edge(&G, u, v);
        }
        has_circle = 0;
        for(u = 1; u <= G.n; u++)
                if(!mark[u]){
                        DFS(&G, u, -1);
                        if(has_circle) {
                                List L; make_null(&L);
                                push_back(&L, start);
                                while (end != start){
                                        push_back(&L, end);
                                        end = parent[end];
                                }
                                push_back(&L, start);
                                for(int i = L.size; i >= 1; i--){
                                        printf("%d ", element_at(&L, i));
```

```
                    }
                    return 0;
                }
            }
        printf("-1");
        return 0;
}
```

Đồ thị phân đôi

Kiểm tra đồ thị phân đôi

```
int color[MAX_N];// -1 có nghia là chua tô màu (chua tham); 0 và 1 là hai màu trái nguoc nhau
int conflict;
void DFS_CheckBipartite(Graph *pG, int u, int cl){
    color[u] = cl;
    for(int v = 1; v <= pG->n; v++){
        if(pG->A[u][v]){
            if(color[v] == -1)
                DFS_CheckBipartite(pG, v, 1 - cl);
            else if(color[v] == color[u])
                conflict = 1;
        }
    }
}

int main(){
    // freopen("dt.txt", "r", stdin);
    int n, m;
    scanf("%d%d", &n, &m);
    Graph G;
    init_graph(&G, n);
    for(int i = 0; i< m; i++){
        int u, v;
        scanf("%d%d", &u, &v);
        add_edge(&G, u, v);
    }
   conflict = 0;
   for(int u = 1; u <= n; u++)
        color[u] = -1;

   for(int u = 1; u <= n; u++)
        if(color[u] == -1)
            DFS_CheckBipartite(&G, u, 0);
    if(conflict)
        printf("NO");
    else printf("YES");
    return 0;
}
```

In ra mỗi bên của đồ thị phân đôi

//Hàm DFS_CheckBipartite

```
int main(){
    // freopen("dt.txt", "r", stdin);
    int n, m;
    scanf("%d%d", &n, &m);
    Graph G;
    init_graph(&G, n);
    for(int i = 0; i< m; i++){
        int u, v;
        scanf("%d%d", &u, &v);
        add_edge(&G, u, v);
    }
   conflict = 0;
   for(int u = 1; u <= n; u++)
        color[u] = -1;

   for(int u = 1; u <= n; u++)
        if(color[u] == -1)
            DFS_CheckBipartite(&G, u, 0);
    if(conflict)
        printf("IMPOSSIBLE");
    else {
        for(int u = 1; u <= n; u++)
            if(color[u] == 0) printf("%d ", u);
        printf("\n");
        for(int u = 1; u <= n; u++)
            if(color[u] == 1) printf("%d ", u);
    }
    return 0;
}
```

Giải thuật Tarjan – Kiểm tra đồ thị liên thông mạnh

Kiểm tra đồ thị liên thông mạnh:

```
Stack S;
int cnt, scc_cnt;
int num[MAX_N], min_num[MAX_N];
int on_stack[MAX_N] ={0};
void SCC(Graph *pG, int u){
        num[u] = min_num[u] = ++cnt;
        push(&S, u);
        on_stack[u] = 1;
        for(int v = 1; v <= pG->n; v++){
                if(pG->A[u][v]){
                        if(num[v] < 0){
                                SCC(pG, v);
                                min_num[u] = min(min_num[u], min_num[v]);
                        }
                        else if(on_stack[v]){
                                min_num[u] = min(min_num[u], num[v]);
                        }
                }
        }
```

```
        if (num[u] == min_num[u]){
                scc_cnt++;
                int w;
                do {
                        w = top(&S);
                        pop(&S);
                        on_stack[w] = 0;
                } while (w != u);
        }
}

int main(){
        Graph G;
        int n, m;
        scanf("%d%d", &n, &m);
        init_graph(&G, n);
        for(int i = 0; i < m; i++){
                int u, v;
                scanf("%d%d", &u,&v);
                add_edge(&G, u, v);
        }
        for(int u = 1; u <= G.n; u++)
                num[u] = -1;
        make_null_stack(&S);
        SCC(&G, 1);
        int visited_all = 1;
    for(int i = 1; i <= n; i++) {
      if(num[i] == -1) {
        visited_all = 0;
        break;
      }
    }
    if(visited_all && scc_cnt == 1)
        printf("STRONG CONNECTED");
    else printf("DISCONNECTED");
        return 0;
}
```

Đếm số bộ phận liên thông mạnh

```
Stack S;
int cnt, scc_cnt;
int num[MAX_N], min_num[MAX_N];
int on_stack[MAX_N] ={0};
void SCC(Graph *pG, int u){
        num[u] = min_num[u] = ++cnt;
        push(&S, u);
        on_stack[u] = 1;
        for(int v = 1; v <= pG->n; v++){
                if(pG->A[u][v]){
                        if(num[v] < 0){
                                SCC(pG, v);
```

```
                        min_num[u] = min(min_num[u], min_num[v]);
                    }
                    else if(on_stack[v]){
                        min_num[u] = min(min_num[u], num[v]);
                    }
                }
            }
            if (num[u] == min_num[u]){
                    scc_cnt++;
                    int w;
                    do {
                        w = top(&S);
                        pop(&S);
                        on_stack[w] = 0;
                    } while (w != u);
            }
}

int main(){
        Graph G;
        int n, m;
        scanf("%d%d", &n, &m);
        init_graph(&G, n);
        for(int i = 0; i < m; i++){
                int u, v;
                scanf("%d%d", &u,&v);
                add_edge(&G, u, v);
        }
        for(int u = 1; u <= G.n; u++)
                num[u] = -1;
        make_null_stack(&S);
        for(int u = 1; u <= G.n; u++)
                if(num[u] == -1){
                        SCC(&G, u);
                }
   printf("%d", scc_cnt);
        return 0;
}
```

Tìm BPLT mạnh có số đỉnh lớn nhất:

```
Stack S;
int cnt, res;
int num[MAX_N], min_num[MAX_N];
int on_stack[MAX_N] ={0};
void SCC(Graph *pG, int u){
        num[u] = min_num[u] = ++cnt;
        push(&S, u);
        on_stack[u] = 1;
        for(int v = 1; v <= pG->n; v++){
                if(pG->A[u][v]){
                        if(num[v] < 0){
                                SCC(pG, v);
```

```
                        min_num[u] = min(min_num[u], min_num[v]);
                    }
                    else if(on_stack[v]){
                        min_num[u] = min(min_num[u], num[v]);
                    }
                }
            }
            if (num[u] == min_num[u]){
                    int w, cnt_v = 0;
                    do {
                        cnt_v++;
                        w = top(&S);
                        pop(&S);
                        on_stack[w] = 0;
                    } while (w != u);
                    res = max(cnt_v, res);
            }
}

int main(){
        Graph G;
        int n, m;
        scanf("%d%d", &n, &m);
        init_graph(&G, n);
        for(int i = 0; i < m; i++){
                int u, v;
                scanf("%d%d", &u,&v);
                add_edge(&G, u, v);
        }
        for(int u = 1; u <= G.n; u++)
                num[u] = -1;
        make_null_stack(&S);
        for(int u = 1; u <= G.n; u++)
                if(num[u] == -1){
                        SCC(&G, u);
                }
    printf("%d", res);
        return 0;
}
```

Đường đi ngắn nhất

Khai báo cấu trúc Ma trận trọng số

```
#include <stdio.h>
#define MAX_N 500
#define oo 10000000
```

1. Khai báo cấu trúc Graph

```
typedef struct{
	int W[MAX_N][MAX_N];
	int n,m;
} Graph;
```

2. Khởi tạo đồ thị

```
void init_graph(Graph *pG, int n){
	pG->n = n;
	pG->m = 0;
	for(int i = 1; i <= n; i++)
		for(int j = 1; j <= n; j++)
			pG->W[i][j] = oo;//Do Moore Dijkstra ap dung do thi trong so "khong
am"
}
```

3. Hàm thêm cạnh

Đồ thị có hướng	Đồ thị vô hướng

`void add_edge(Graph *pG, int u, int v, int w){`
`pG->W[u][v] = w;`
`}`
	`void add_edge(Graph *pG, int u, int v, int w){`
	`pG->W[u][v] = pG->W[v][u] = w;`
	`}`

Thuật toán Moore-Dijkstra

Code thuật toán

```
#define MAX_N 500
#define oo 10000000
#define min(x,y) (x>y)?y:x
int p[MAX_N], mark[MAX_N], pi[MAX_N];

void Dijkstra(Graph *pG, int s){
	for(int u = 1; u <= pG->n; u++){
		pi[u] = oo;
		p[u] = -1;
		mark[u] = 0;
	}
	pi[s] = 0;
	for(int i = 1; i <= pG->n - 1; i++){
		int min_pi = oo;
		int u = -1;
	for(int j = 1; j <= pG->n; j++)
	    if (!mark[j] && pi[j] < min_pi) {
	       min_pi = pi[j];
```

```
            u = j;
        }
                if(u == -1) break;

                mark[u] = 1;

                for(int v = 1; v <= pG->n; v++)
                        if(pG->W[u][v] != oo && !mark[v]){
                                if(pi[u] + pG->W[u][v] < pi[v]){
                                    pi[v] = pi[u] + pG->W[u][v];
                                    p[v] = u;
                                }
                        }
        }
}
```

Bắt đầu từ đỉnh 1

```
int main(){
        int n,m;
        Graph G;
        scanf("%d%d", &n, &m);
        init_graph(&G, n);
        for(int i = 0; i < m; i++){
                int u, v, w;
                scanf("%d%d%d", &u, &v, &w);
                add_edge(&G, u, v, w);
        }
        Dijkstra(&G, 1);
        for(int i = 1; i <= n; i++)
                printf("pi[%d] = %d, p[%d] = %d\n", i, pi[i], i, p[i]);
        return 0;
}
```

Bắt đầu từ đỉnh s nhập từ bàn phím

```
int main(){
        int n,m;
        Graph G;
        scanf("%d%d", &n, &m);
        init_graph(&G, n);
        for(int i = 0; i < m; i++){
                int u, v, w;
                scanf("%d%d%d", &u, &v, &w);
                add_edge(&G, u, v, w);
        }
        int s; scanf("%d", &s);
        Dijkstra(&G, s);
        for(int i = 1; i <= n; i++)
                printf("pi[%d] = %d, p[%d] = %d\n", i, pi[i], i, p[i]);
        return 0;
}
```

In đường đi từ 1 đến n

```
int path[MAX_N], length;
void find_Short_Path(Graph *pG, int s, int t){
	int end = t, idx = 0;
	while (s != end){
		path[idx++] = end;
		end = p[end];
	}
	path[idx] = s;
	length = idx;
}

int main(){
	int n,m;
	Graph G;
	scanf("%d%d", &n, &m);
	init_graph(&G, n);

	for(int i = 0; i < m; i++){
		int u, v, w;
		scanf("%d%d%d", &u, &v, &w);
		add_edge(&G, u, v, w);
	}
	int s, t;
	scanf("%d%d", &s, &t);
	Dijkstra(&G, s);
	find_Short_Path(&G, s, t);
	for(int i = length; i >= 0; i--){
		if(i != 0){
			printf("%d -> ", path[i]);
		}
		else printf("%d", path[i]);
	}
	return 0;
}
```

Khai báo cấu trúc DS Cạnh có trọng số

```
#include <stdio.h>
#define MAX_N 500
#define MAX_M 1000
#define oo 10000000
```

1. Khai báo cấu trúc cạnh

```
typedef struct{
	int u, v, w;
} Edge;
```

2. Khai báo cấu trúc Graph

```
typedef struct {
	int n, m;
	Edge edges[MAX_N];
```

```
}Graph;
```

3. Khởi tạo đồ thị

```
void init_graph(Graph *pG, int n, int m){
	pG->m = 0;
	pG->n = n;
}
```

4. Hàm thêm cạnh

```
void add_edge(Graph *pG, int u, int v, int w){
	pG->edges[pG->m].u = u;
	pG->edges[pG->m].v = v;
	pG->edges[pG->m].w = w;
	pG->m++;
}
```

Thuật toán Bellman Ford:

Code thuật toán

```
int p[MAX_N], mark[MAX_N], pi[MAX_N];

void BellmanFord(Graph *pG, int s){
	for(int u = 1; u <= pG->n; u++){
		pi[u] = oo;
	}
	pi[s] = 0;
	p[s] = -1;
	int u, v, w;
	for(int it = 1; it < pG->n; it++){
		for(int k = 0; k < pG->m; k++){
			u = pG->edges[k].u;
			v = pG->edges[k].v;
			w = pG->edges[k].w;
			if(pi[u] == oo)
				continue;
			if(pi[u] + w < pi[v]){
				pi[v] = pi[u] + w;
				p[v] = u;
			}
		}
	}
}
```

In đường đi từ S đến T:

```
//Hàm thuật toán BellmanFord
int path[MAX_N], length;
void find_Short_Path(Graph *pG, int s, int t){
	int end = t, idx = 0;
	while (s != end){
		path[idx++] = end;
		end = p[end];
```

```
	}
	path[idx] = s;
	length = idx;
}

int main(){
	int n,m;
	Graph G;
	scanf("%d%d", &n, &m);
	init_graph(&G, n,m);

	for(int i = 0; i < m; i++){
		int u, v, w;
		scanf("%d%d%d", &u, &v, &w);
		add_edge(&G, u, v, w);
	}
	int s, t;
	scanf("%d%d", &s, &t);
	BellmanFord(&G, s);
	find_Short_Path(&G, s, t);
	for(int i = length; i >= 0; i--){
		if(i != 0){
			printf("%d -> ", path[i]);
		}
		else printf("%d", path[i]);
	}
	return 0;
}
```

Kiểm tra và in ra chu trình âm

```
int p[MAX_N], mark[MAX_N], pi[MAX_N], negative_cycle;
int cycle_len, cycle[MAX_N];
void BellmanFord(Graph *pG, int s){
	for(int u = 1; u <= pG->n; u++){
		pi[u] = oo;
	}
	pi[s] = 0;
	p[s] = -1;
	int u, v, w;
	for(int it = 1; it < pG->n; it++){
		for(int k = 0; k < pG->m; k++){
			u = pG->edges[k].u;
			v = pG->edges[k].v;
			w = pG->edges[k].w;
			if(pi[u] == oo)
				continue;
			if(pi[u] + w < pi[v]){
				pi[v] = pi[u] + w;
				p[v] = u;
			}
		}
	}
```

```
    int x = -1;
    for(int k = 0; k < pG->m; k++){
        int u = pG->edges[k].u;
        int v = pG->edges[k].v;
        int w = pG->edges[k].w;
        if(pi[u] == oo) continue;
        if(pi[u] + w < pi[v]){
            p[v] = u;
            x = v;
            negative_cycle = 1;
            break;
        }
    }
    if(!negative_cycle) return;

            for(int i = 1; i <= pG->n; i++){
                x = p[x];
                if (x == -1) {
                                negative_cycle = 0;
                                cycle_len = 0;
                                return;
                        }
            }

    int cur = x;
    do{
        cycle[cycle_len++] = cur;
        cur = p[cur];
    } while(cur != x);

    cycle[cycle_len++] = x;
}

int main(){
            int n,m;
            Graph G;
            scanf("%d%d", &n, &m);
            init_graph(&G, n,m);

            for(int i = 0; i < m; i++){
                        int u, v, w;
                        scanf("%d%d%d", &u, &v, &w);
                        add_edge(&G, u, v, w);
            }
            int s;
            scanf("%d", &s);
            BellmanFord(&G, s);
            if(negative_cycle) {
                printf("YES\n");
                for(int i = 0; i < cycle_len; i++){
                    printf("%d", cycle[i]);
                    if(i < cycle_len - 1) printf(" -> ");
```

```
            }
        }
        else printf("NO");
        return 0;
}
```

Thuật toán Topo & Ứng Dụng

Thuật toán Sắp xếp Topo:

* BFS:

```
int d[MAX_N];
void topo_sort(Graph *pG, List *pL){
        for(int u = 1; u <= pG->n; u++){
                for(int v = 1; v <= pG->n; v++)
                        if(pG->A[v][u])
                                d[u]++;
        }
        Queue Q;
        make_null_queue(&Q);

        for(int u = 1; u <= pG->n; u++)
                if(d[u] == 0)
                        enqueue(&Q, u);

        make_null_list(pL);

        while(!empty_queue(&Q)){
                int u = front(&Q);
                dequeue(&Q);
                push_back(pL, u);
                for(int v = 1; v <= pG->n; v++)
                        if(pG->A[u][v]){
                                d[v]--;
                                if(d[v] == 0)
                                        enqueue(&Q, v);
                        }
        }
}
```

* DFS đệ quy:

```
int d[MAX_N], mark[MAX_N];
void topo_sort(Graph *pG, int u, List *pL){
        mark[u] = 1;
        for(int v = 1; v <= pG->n; v++)
                if(pG->A[u][v] && !mark[v]){
                        topo_sort(pG, v, pL);
                        d[v]--;
                }
        push_back(pL, u);
```

```
}

void reverse_List(List *pL){
	for(int i = 0; i < pL->size/2; i++){
		int tmp = pL->data[i];
		pL->data[i] = pL->data[pL->size- i - 1];
		pL->data[pL->size - i - 1] = tmp;
	}
}
```

Thuật toán xếp hạng:

```
int d[MAX_N], r[MAX_N];
void rank(Graph *pG){
	for(int u = 1; u <= pG->n; u++){
		for(int v = 1; v <= pG->n; v++)
			if(pG->A[v][u])
				d[u]++;
	}
	List S1, S2;

	make_null_list(&S1);
	for(int u = 1; u <= pG->n; u++)
		if(d[u] == 0)
			push_back(&S1, u);

	int k = 0;
	while (S1.size > 0){
		make_null_list((&S2));
		for(int p = 1; p <= S1.size; p++){
			int u = element_at(&S1, p);
			r[u] = k;

			for(int v = 1; v <= pG->n; v++)
				if(pG->A[u][v]){
					d[v]--;
					if(d[v] == 0)
						push_back(&S2, v);
				}
		}
		S1 = S2;
		k++;
	}
}
```

Quản lý công việc

```
int indeg[MAX_N];
void topo_sort(Graph *pG, List *pL){
```

```
	for(int u = 1; u <= pG->n; u++){
		for(int v = 1; v <= pG->n; v++)
			if(pG->A[v][u])
				indeg[u]++;
	}
	Queue Q;
	make_null_queue(&Q);

	for(int u = 1; u <= pG->n; u++)
		if(indeg[u] == 0)
			enqueue(&Q, u);

	make_null_list(pL);

	while(!empty_queue(&Q)){
		int u = front(&Q);
		dequeue(&Q);
		push_back(pL, u);
		for(int v = 1; v <= pG->n; v++)
			if(pG->A[u][v]){
				indeg[v]--;
				if(indeg[v] == 0)
					enqueue(&Q, v);
			}
	}
}

int d[MAX_N];
int main(){
	int n;
	scanf("%d", &n);
	Graph G;
	init_graph(&G, n+2);
	int alpha = n + 1; int beta = n+2;
	for(int u = 1; u <= n; u++){
		int x;
		scanf("%d", &d[u]);
		do{
			scanf("%d", &x);
			if (x > 0)
				add_edge(&G, x, u);
		} while(x > 0);
	}

	for(int u = 1; u <= n; u++){
		int deg_neg = 0;
		for(int x = 1; x <= n; x++){
			if (G.A[x][u]){
				deg_neg++;
			}
		}
		if(deg_neg == 0){
```

```
                add_edge(&G, alpha, u);
            }
        }

        for(int u = 1; u <= n; u++){
            int deg_pos = 0;
            for(int v = 1; v <= n; v++){
                if(G.A[u][v]){
                    deg_pos++;
                }
            }
            if(deg_pos == 0)
                add_edge(&G, u, beta);
        }

        List L;
        topo_sort(&G, &L);

        int t[MAX_N];
        t[alpha] = 0;

        for(int j = 2; j <= L.size; j++){
            int u = element_at(&L, j);
            t[u] = -oo;
            for(int x = 1; x <= G.n; x++){
                if (G.A[x][u]){
                    t[u] = max(t[u], t[x] + d[x]);
                }
            }
        }

        int T[MAX_N];
        T[beta] = t[beta];

        for(int j = L.size - 1; j >= 1; j--){
                int u = element_at(&L, j);
                T[u] = +oo;
                for(int v = 1; v <= G.n; v++)
                    if(G.A[u][v]){
                        T[u] = min(T[u], T[v] - d[u]);
                    }
        }

        for(int i = 1; i <= n; i++){
            printf("%d %d\n", t[i], T[i]);
        }
        return 0;
}
```

Cây khung nhỏ nhất + Luồng cực đại

Thuật toán Kruskal

Khai báo danh sách cung có trọng số:

```
#include <stdio.h>

typedef struct {
   int x,y,w;
}Edge;

typedef struct {
  Edge edges[100];
  int n, m;
}Graph;

void init_Graph(Graph *pG, int n){
  pG->n = n;
  pG->m = 0;
}
void add_edge(Graph *pG, int u, int v, int z){
  pG->edges[pG->m].x = u;
  pG->edges[pG->m].y = v;
  pG->edges[pG->m].w = z;
  pG->m++;
}
```

Thuật toán Kruskal:

```
int parent[100];
int find_root(int u){
  while (parent[u] != u){
    u = parent[u];
  }
  return u;
}

void bubble_sort(Graph *pG){
  for (int i = 0 ; i < pG->m-1; i++){
    for (int j =pG->m-1; j > 0;j--){
      if (pG->edges[j].w < pG->edges[j-1].w){
        Edge tmp = pG->edges[j] ;
        pG->edges[j] = pG->edges[j-1];
        pG->edges[j-1]= tmp;
      }
    }
  }
}

int krusal(Graph *pG ,Graph *T){
  init_Graph(T, pG->n);
```

```
  bubble_sort(pG);
  int n = pG->n;
  int m = pG->m;
  for (int i = 1; i <= n ; i++){
    parent[i] = i;
  }
  int sum = 0;
  for (int i = 0; i < m; i++){
    int u = pG->edges[i].x;
    int v = pG->edges[i].y;
    int w = pG->edges[i].w;
    int root_u = find_root(u);
    int root_v = find_root(v);
    if (root_u != root_v){
      if (u < v ) add_edge(T, u, v, w);
      else add_edge(T, v, u, w);
      parent[root_v] = root_u;
      sum += w;
    }
  }
  return sum;
}

int main(){
  int n, m;
  scanf("%d%d", &n, &m);
  Graph G; init_Graph(&G, n);
  for (int i = 1; i <= m; i++) {
    int x, y, w;
    scanf("%d%d%d", &x, &y, &w);
    add_edge(&G, x, y,w);
  }
  Graph T;
  int total = krusal(&G, &T);
  printf("%d\n", total);
  for (int  i =0 ; i < T.m; i++){
    printf("%d %d %d\n", T.edges[i].x, T.edges[i].y, T.edges[i].w);
  }
  return 0;
}
```

Thuật toán Prim

```
//Khai báo cấu trúc ma trận trọng số
int parent[100];
int pi[100];
int mark[100];

int Prim(Graph *pG, int s ,Graph *pT){
  int n = pG->n;
  for (int i = 1; i <= n; i++){
    pi[i] = oo;
```

```
    parent[i] = -1;
    mark[i] = 0;
  }
  pi[s] = 0;
  for (int it = 1; it <= n; it ++){
    int u  = -1;
    int val = oo;
    for (int i = 1; i <= n ; i++){
      if (!mark[i] && pi[i] < val){
        val = pi[i];
        u = i;
      }
    }
    mark[u] = 1;
    for (int v = 1; v <= n; v++){
      if (pG->A[u][v] && !mark[v] && pi[v] > pG->A[u][v]){
        pi[v] = pG->A[u][v];
        parent[v] = u;
      }
    }
  }
  init_Graph(pT, n);
  int sum = 0 ;
  for (int u = 1; u <= n ; u++){
    if (parent[u] != -1){
      sum+= pG->A[parent[u]][u];
      add_edge(pT, parent[u], u, pG->A[parent[u]][u]);
    }
  }
  return sum;
}

int main(){
  int n, m;
  scanf("%d%d", &n, &m);
  Graph G; init_Graph(&G, n);
  for (int i = 1; i <= m; i++) {
    int x, y,w;
    scanf("%d%d%d", &x, &y, &w);
    add_edge(&G, x, y,w);
  }
  Graph T;
  int sum = Prim(&G, 1, &T);
  printf("%d\n", sum);
  for (int u = 1; u <= n; u++){
    for (int v =1 ;v <= n ;v ++){
      if (T.A[u][v] && u < v){
        printf("%d %d %d\n",u , v, T.A[u][v]);
      }
    }
  }
  return 0;
```

```
}
```

Check luồng hợp lệ

```
#include <stdio.h>

int in[100];
int out[100];

int main(){
  int n, m; scanf("%d%d", &n,&m);
  int check = 1;
  for (int i =1; i <= m ; i++){
    int u, v ,c, f;
    scanf("%d%d%d%d", &u, &v, &c, &f);
    out[u] += f;
    in[v] += f;
    if (f > c) check = 0;
  }
  if (out[1] != in[n]) check = 0;
  for (int i = 2; i <= n-1 ; i++){
    if (in[i] !=out[i]) check = 0;
  }
  if (check == 0) printf("NO");
  else printf("YES");

  return 0;
}
```

Luồng cực đại - PP FullKerson

```
#include <stdio.h>
#define MIN(a,b) (a>b)?(b):(a)
#define oo 100000
typedef int ElementType;

typedef struct{
  int C[100][100];
  int F[100][100];
  int n;
} Graph;

typedef struct{
  int dir;
  int p;
  int sigma;
}Label;

Label labels[100];
void init_flow(Graph *pG){
  for (int i = 1; i <= pG->n; i++){
    for (int j = 1; j <= pG->n; j++){
```

```
      pG->F[i][j] = 0;
    }
  }
}

//Khai báo cấu trúc hàng đợi
….
int FordFullkerson(Graph *pG ,int s, int t){
  int n = pG->n;
  init_flow(pG);
  int max_flow = 0;
  do {
    for (int i = 1; i <= n ; i++){
      labels[i].dir = 0;
    }
    labels[s].dir = 1;
    labels[s].p = -1;
    labels[s].sigma = oo;
    Queue Q; makenull_Queue(&Q);
    enqueue(&Q, s);
    int found = 0;
    while (!empty_Queue(&Q)){
      int u = front(&Q); dequeue(&Q);
      for (int  v= 1; v <= n ; v++){
        if (pG->C[u][v] && labels[v].dir == 0 && pG->F[u][v] < pG->C[u][v]){
          labels[v].dir = 1;
          labels[v].p = u;
          labels[v].sigma = MIN(labels[u].sigma, pG->C[u][v] - pG->F[u][v]);
          enqueue(&Q, v);
        }
      }
       if (labels[t].dir != 0){
        found = 1;
        break;
      }
      for (int  v= 1; v <= n ; v++){
        if (pG->C[v][u] && labels[v].dir == 0 && pG->F[u][v] > 0){
          labels[v].dir = -1;
          labels[v].p = u;
          labels[v].sigma = MIN(labels[u].sigma, pG->F[u][v]);
          enqueue(&Q, v);
        }
      }
      if (labels[t].dir != 0) {
        found = 1;
        break;
      }
    }
    if (found == 1){
      int sigma = labels[t].sigma;
      int u = t;
      while (u != s){
```

```
        int v = labels[u].p;
        if (labels[v].dir == 1){
          pG->F[v][u] += sigma;
        }
        else{
          pG->F[u][v] -= sigma;
        }
        u = v;
      }
      max_flow += sigma;
    }else break;
  }while (1);
  return max_flow;
}

int main(){
  Graph G;
  int n, m; scanf("%d%d", &n, &m);
  G.n = n;
  for (int i = 1; i <= n ; i++){
    for (int j = 1; j <= n; j++){
      G.C[i][j] = 0;
    }
  }
  for (int i = 1 ; i <= m; i++){
    int u,v,w;
    scanf("%d%d%d", &u, &v ,&w);
    G.C[u][v] = w;
  }
  int max_flow = FordFullkerson(&G, 1, n);
  printf("Max flow: %d\n", max_flow);
  printf("S: ");
  for (int i = 1; i <=n ; i++){
    if (labels[i].dir != 0) printf("%d ", i);
  }
  printf("\nT: ");
  for (int i = 1; i <=n ; i++){
    if (labels[i].dir == 0) printf("%d ", i);
  }
  return 0;
}
```

BONUS: Các cấu trúc dữ liệu cần thiết

```
#define MAX_N 100
```

List

```
typedef struct{
	int size;
	int data[MAX_N];
}List;

void make_null_list(List *pL){
	pL->size = 0;
}

void push_back(List *pL, int x){
	pL->data[pL->size++] = x;
}

int element_at(List *pL, int p){
	return pL->data[p-1];
}
```

Queue

```
typedef struct{
	int front, rear;
	int data[MAX_N];
}Queue;

void make_null_queue(Queue *pQ){
	pQ->front = 0;
	pQ->rear = -1;
}

void enqueue(Queue *pQ, int x){
	pQ->data[++pQ->rear] = x;
}

void dequeue(Queue *pQ){
	pQ->front++;
}

int front(Queue *pQ){
	return pQ->data[pQ->front];
}

int empty_queue(Queue *pQ){
	return pQ->front > pQ->rear;
}
```

Stack

```
typedef struct{
	int data[MAX_N];
	int top_idx;
} Stack;

void make_null_stack(Stack *pS){
	pS->top_idx = -1;
}

int empty(Stack *pS){
	return pS->top_idx == -1;
}

void push(Stack *pS, int x){
	pS->top_idx++;
	pS->data[pS->top_idx] = x;
}

int top(Stack *pS){
	return pS->data[pS->top_idx];
}

void pop(Stack *pS){
	pS->top_idx--;
}
```